

AutoBench

Architecture & Design Overview

A pure-Python, HTTP-only service that deploys and evaluates agent benchmarks across multiple cluster-specific Rossoctl instances.

Last modified 2026-09-29 20:59 EDT

Agenda

What this deck covers, in order

1

Overview & key design decisions

what the Service is, and the seven choices that shape it

2

Auto-benchmarking design motif

the shape of the system, before the wiring detail

3

Architecture

client, Service, and the per-cluster instance selected by JWT iss

4

Architecture with workload specific components

4.1 inside the workload: sidecar, user simulator, IBAC judge · 4.2 how interception is wired

5

The two-token auth model

why the caller's token is never forwarded upstream

6

Benchmark catalog & run lifecycle

6.1 the three benchmarks and the REST flow that drives them · 6.2 what each image bakes in · 6.3 one task end to end · 6.4 who decides what happens inside it

7

The three benchmarks — what they measure

7.1 the difficulty ladder · 7.2 what each stresses · 7.3 the three compared · 7.4 the traps · 7.5 picking one · 7.6 what it costs · 7.7 the rate card · 7.8–7.9 six efficiency figures

8

The canonical 12-run matrix

8.1 what the 12 runs parameterize · 8.2 what each band established · 8.3 what they measured

9

Cross-platform & plugin overhead

9.1 OpenShift vs KinD, like-for-like · 9.2 what AuthBridge costs

10

What the Service can & cannot enact

the HTTP-only boundary, made explicit

1. Overview & Key Design Decisions

What it is

Deploys a benchmark's MCP tool + A2A agent, runs the evaluation, reads results, and exports them — all over HTTP.

Objectives

- Automate benchmarking lifecycle operations
- Cross-user/cluster sharing of benchmark run results
- Easy to configure in-cluster & cross-cluster benchmark runs
- Asynchronous parallel benchmark runs
- Secure, scalable, auditable & resilient

3 benchmarks

- gsm8k — single-turn: one prompt in, one answer out, no dialogue partner
- tau2 — multi-turn: a server-side user simulator LLM plays the customer
- appworld — long-horizon: many API calls across simulated apps

Client contract

- Point at a hostname, send Authorization: Bearer <caller JWT> — same from kind to prod

Key design decisions

Pure-Python, HTTPS-only to Rossoctl

No shell-out: smaller image, no injection surface, structured errors, testable.

Two-token auth model

Caller JWT attributes + routes only (never forwarded); Service mints its own benchmarker (ROPC) token to Rossoctl.

iss is the trust anchor

The JWT issuer selects the per-instance config — no separate instance argument, so no confused-deputy escape.

Per-request ROPC login

Fresh Service token every request — no expiry handling.

iss-keyed per-instance config

One file per issuer: Rossoctl URL, benchmarker cred, MLflow/S3, optional Keycloak backchannel + workload route templates.

Enact only what the API allows

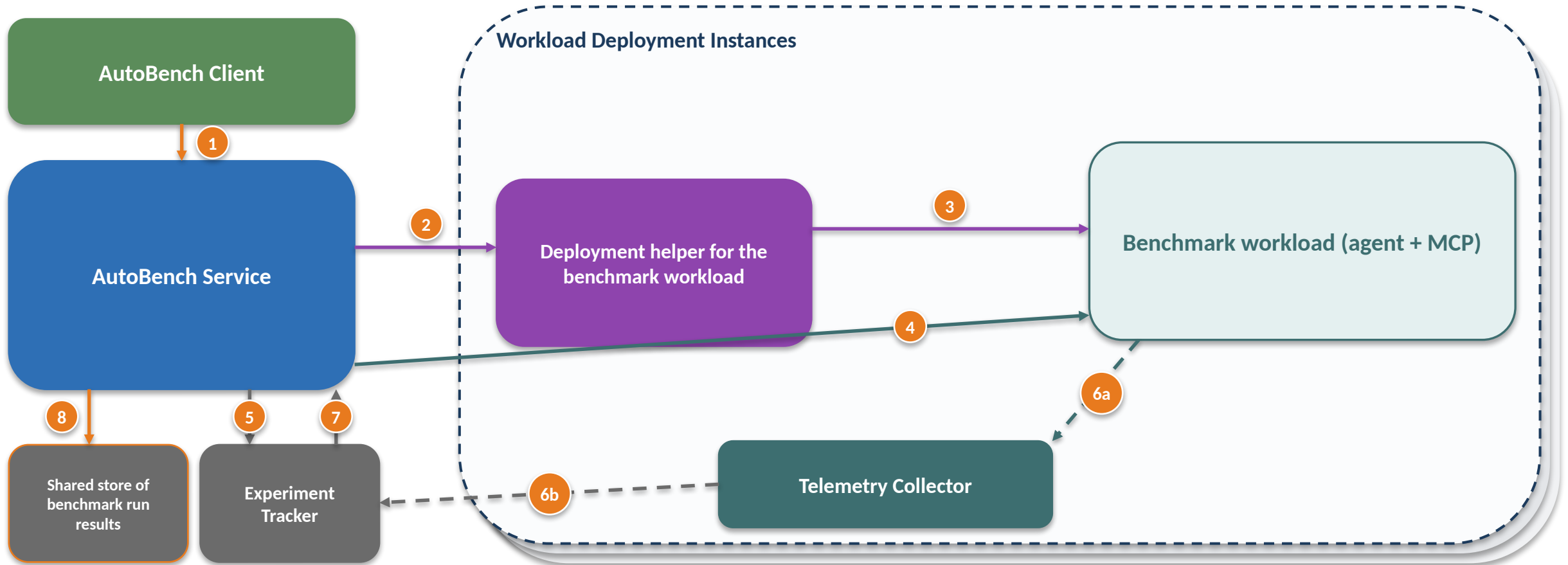
Deploy/run/report/export — now including AuthBridge plugin presets (layer-3). Only workload Secrets remain out-of-band: prechecked and reported (424), never silently ignored.

MLflow with Service; S3 in the cloud

MLflow is co-located with the Service (per-service traces it emits + reads), not in the workload cluster; S3 is an external cloud service — the shared cross-service sink.

2. Auto-Benchmarking Design Motif

One client, one Service, N workload deployments — and every result ends up in one place

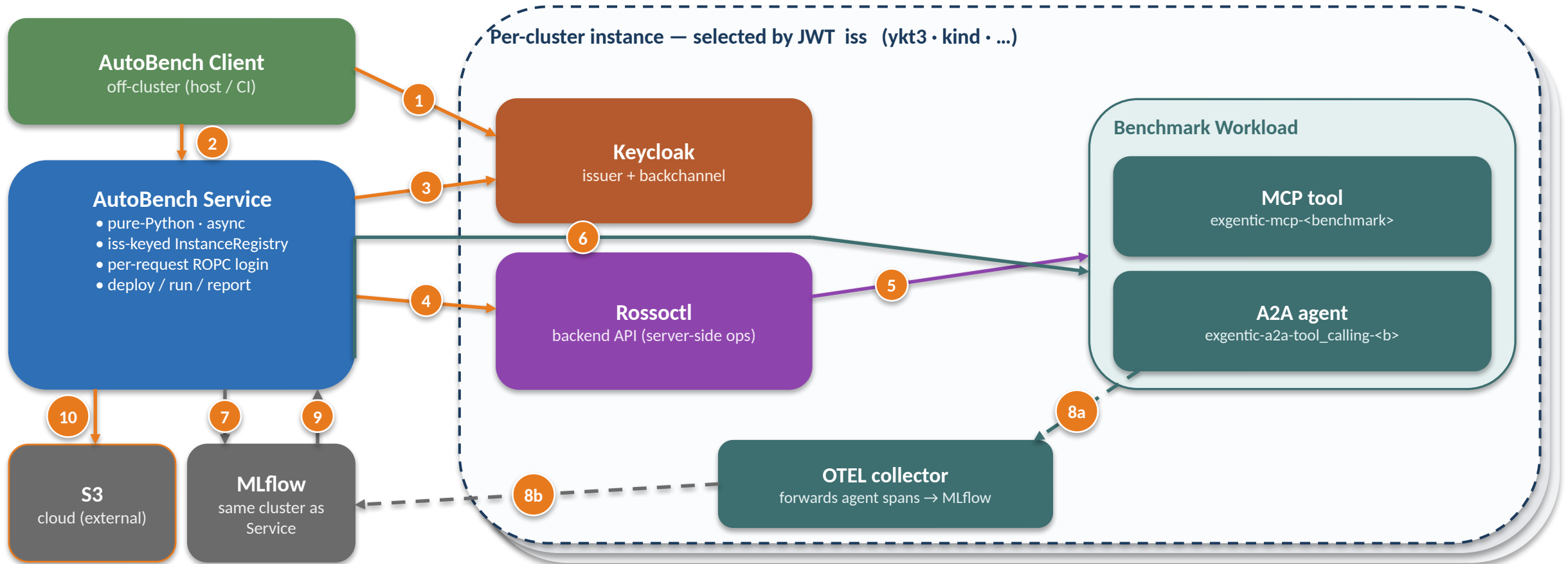


- 1 Client → Service
- 2 Service → deployment helper: create the benchmark workload
- 3 helper → benchmark workload
- 4 Service → benchmark workload: run and monitor execution

- 5 Service → Experiment Tracker: emit the trace
- 6a workload → Telemetry Collector · 6b collector → Tracker
- 7 Experiment Tracker → Service: read the records back
- 8 Service → shared store: export the run's artifacts

3. Implementation Architecture: Kubernetes-Based Benchmark Workloads

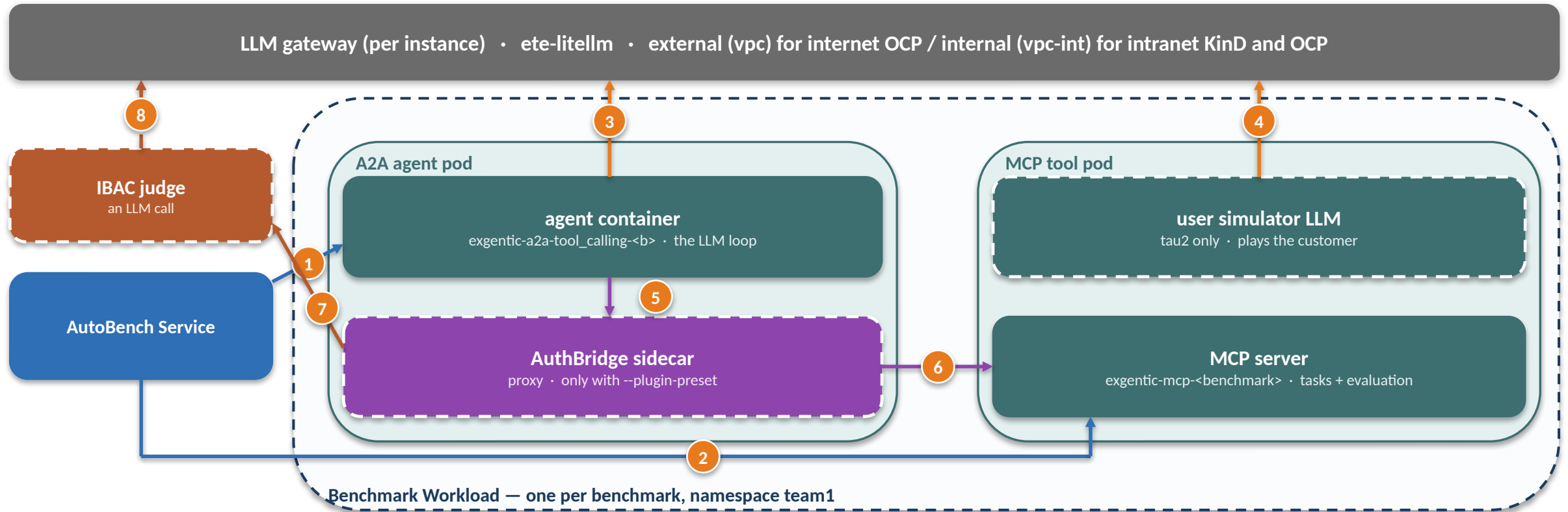
Relations between client, service, cluster-specific Keycloak / Rossoctl / workload, MLflow & S3



- 1 Client logs in to Keycloak (ROPC) → caller JWT
- 2 Client → Service: Authorization: Bearer <caller JWT>
- 3 Service validates JWT (JWKS) + mints its own benchmarker token (ROPC, via backchannel)
- 4 Service → Rossoctl: deploy / list agents + tools
- 5 Rossoctl creates the MCP tool + A2A agent in-cluster
- 6 Service drives the MCP session + the A2A call itself; the agent only issues execute_tool inside that session
- 7 Service emits the Agent.Session trace → MLflow (first)
- 8a A2A agent → OTEL collector · 8b collector → MLflow (optional workload spans — off by default)
- 9 Service reads back Agent.Session traces (MLflow)
- 10 Service exports run.json / report.* (S3)

4.1 Architecture with Workload Specific Components

Inside the Benchmark Workload — what every benchmark has, and what only some of them add



Which components a benchmark actually gets

benchmark	MCP tool image	LLMs per task	tool calls
gsm8k	exgentic-mcp-gsm8k	1 (agent)	~1
tau2	exgentic-mcp-tau2	2 (+ user simulator)	~11
appworld	exgentic-mcp-appworld	1 (agent)	~15
+ any preset	unchanged	+1 IBAC judge call per action	—

- 1 Service → agent: send_prompt, once per task
 - 2 Service → MCP server: list_tasks, create_session, evaluate_session, delete_session (the verdict)
 - 3 agent → gateway: N real chat calls per task
 - 4 tau2 only: the user simulator is a 2nd LLM
 - 5 with a preset: the agent's MCP calls go via the sidecar
 - 6 sidecar → MCP server, once authorized
 - 7 sidecar → judge, per isAction call
 - 8 the judge is itself an LLM call
- Dashed outline = present only in some configurations.
No preset: 5 + 6 become one direct agent → MCP call.

4.2 How Interception Is Wired — and How It Silently Isn't

The sidecar sees traffic only because of two env vars; a plausible config takes it back out of the path

Interception is a blanket forward proxy plus an allowlist — not a tool-aware hook. Three settings decide what the sidecar actually sees:

HTTP_PROXY = HTTPS_PROXY = http://127.0.0.1:8081

the OPERATOR injects this into the agent pod when AuthBridge is enabled — 127.0.0.1:8081 IS the sidecar, so by default every outbound HTTP call the agent makes is captured

no_proxy = LLM gateway, OTEL collector, Keycloak

instance config — the hosts that stay DIRECT. This is the only reason flow 3, the agent's own chat calls, does not go through the sidecar

judge_inference: false

IBAC's own default — even proxied inference is not judged. BOTH this and no_proxy would have to change for the agent's model calls to be authorized

✓ Through the sidecar

- agent → MCP tool calls (flows 5 → 6) — the interception point
- tools/call → isAction=true → one IBAC judge call each (flow 7)
- inbound A2A message/stream → a2a-parser, isAction=false, so the Service's send_prompt is logged but never judged

✗ Direct — the plugins never see it

- agent → LLM gateway (flow 3) — named in no_proxy, and judge_inference: false would exempt it even if proxied
- Service → MCP server (flow 2) — a different pod; the Service does not dial through the agent's proxy
- MCP tool → gateway (flow 4, tau2's user simulator) — different pod

A ready sidecar proves nothing about enforcement — the double bypass we actually shipped:

1. workload_llm.disable_proxy: true makes the SERVICE inject HTTP_PROXY="" first

2. the operator adds a var only when ABSENT → it skips HTTP_PROXY, but still sets HTTPS_PROXY

3. MCP_URL is http:// → tool calls take the empty HTTP_PROXY and go direct

4. no_proxy carried .svc.cluster.local → the same calls are excluded a second time

Result: sidecar ready = True, pipeline ConfigMap rendered, plugins loaded — and the judge called ZERO times, on both clusters.

Verify by COUNTING judge chat completions across a run. IBAC emits no log line of its own, so a healthy sidecar log proves nothing — and initialize / tools/list are isAction=false and legitimately never reach the judge, so a zero count only means something if real tool calls occurred. Working config: leave disable_proxy off, keep the NAMED hosts in no_proxy, drop the WILDCARDS (.svc, .svc.cluster.local).

5. The Two-Token Auth Model

Why the caller's token is never forwarded upstream

Caller JWT (inbound)

Presented by the client as Authorization: Bearer.

- Used ONLY to attribute (preferred_username) and route (iss → instance)
- Signature validated via the issuer's JWKS
- aud / exp deliberately lenient in the dev/ops context
- **NEVER forwarded to Rossoctl**

benchmarker is special:

- A caller JWT with preferred_username == benchmarker authorizes config ops (GET/PUT /config)

Service token (outbound)

Minted by the Service per request via ROPC (password grant) using the instance's benchmarker credential.

- Always fresh → no expiry handling
- Presented to Rossoctl for all cluster-facing ops
- **The Service acts under its OWN identity, not the caller's**

Backchannel split (iss ≠ dialed host):

- When the iss host is unreachable (in-cluster kind), JWKS + token come from a configured backchannel URL; iss is matched, never dialed

Implication: at the Rossoctl / cluster layer every action appears as the Service's identity — so per-user attribution & audit live in the Service, keyed on (iss, preferred_username).

6.1 Benchmark Catalog & Run Lifecycle

Catalog (static registry)

gsm8k

single-turn · needs hf-secret + openai-secret

tau2

multi-turn · user-simulator LLM runs server-side in the MCP pod · raise timeout

appworld

long-horizon across simulated apps · runs e2e · raise task_timeout_seconds

Lifecycle (REST)

POST /benchmarks/{n}/deploy

create MCP tool + A2A agent → 201

GET /benchmarks/{n}/status

poll until tool_ready & agent_ready

POST /benchmarks/{n}/runs

precheck → 202 run_id (409 not deployed · 424 secret missing)

GET .../runs/{id}

poll pending → running → succeeded / failed

GET .../runs/{id}/report

MLflow records + artifacts (409 if MLflow unset)

download from S3

run.json · report.* · token_report.* · span_report.* · manifest.json

6.2 What Each Benchmark Bakes In — and What It Needs From Us

The MCP image carries the benchmark itself; only the credentials and quirk overrides come from tool_env

	MCP tool image	what the image bakes in	what tool_env must add
gsm8k	exgentic-mcp-gsm8k	the HuggingFace loader, pinned to main/test — those 1,319 rows fetched at pod startup	HF_TOKEN (from hf-secret), plus EXGENTIC_SET_BENCHMARK_RUNNER=direct
tau2	exgentic-mcp-tau2	tau2-bench at tag v0.1.3 + its retail domain (114 tasks), and a user-simulator LLM	OPENAI_API_KEY (from openai-secret) + EXGENTIC_SET_BENCHMARK_ACTION_TIMEOUT=1000 — the simulator makes its own inference calls (flow 4)
appworld	exgentic-mcp-appworld	the whole app-suite sandbox at upstream commit edc96012, under a CUSTOM tool-per-API adapter; test_normal split	nothing beyond BENCHMARK_NAME — upstream's .env.appworld is explicitly empty

One agent image serves all three

exgentic-a2a-tool_calling:latest is the only key in every agents={...} map; per benchmark it only gains a -<benchmark> name suffix. So the benchmark lives in the MCP pod, and the subject under test is the same binary every time.

The LLM base is never baked in

OPENAI_API_BASE is injected per deploy from the instance's workload_llm.api_base, and tau2's simulator model from the run's model — a deploy with no gateway configured is rejected with 422 rather than defaulting.

Two traps in that last column: appworld REJECTS the same action-timeout override tau2 needs and crashes at startup with "Unknown benchmark override 'action_timeout'" — the env is per-benchmark, not a shared default. And hf-secret must EXIST for gsm8k even though the dataset is public: without it the MCP pod sits in CreateContainerConfigError and the agent crash-loops. Missing secrets surface as a 424 on the run precheck, naming exactly what to provision.

6.3 One Task, End to End

The same system in time — and every model call and tool call lives inside ONE A2A turn

who	step	span it appears as
Service	create_session(task_id) → MCP pod	MCP.CreateSession
Service	send_prompt(task text; session_id in A2A request metadata) → agent — ONE message, once per task	Agent.Call
agent	connect_mcp — tools/list → MCP pod (per task, not per pod; ~27–38 ms)	connect_mcp
agent	create_agent (6.7 ms warm, up to 6.5 s on a cold pod)	create_agent
agent	initial_observation — local, no I/O, ~50 μs	execute_tool initial_observation (never counted)
agent	LOOP: chat(model) → execute_tool(...) → MCP pod → observation → chat → ...	chat <model> · execute_tool <tool>
agent	returns its final message, when the model asks for no further tool	(the runner discards the text)
Service	evaluate_session → MCP pod: the verdict, read off the session's final state	Evaluator.Evaluate
Service	delete_session → MCP pod, in a finally	(no span)

One A2A turn per task

Everything the agent does happens inside that single streaming request. Its POST / span brackets it, so Agent.Call – POST / is the Service's own cost: 16.9 ms of a 10.4 s gsm8k task, 13.2 ms of a 70.3 s tau2 one.

The Service issues no tool call

Its MCP traffic is list_tasks / create_session / evaluate_session / delete_session. execute_tool is the AGENT's outbound call, named by the agent — the Service never learns a tool's name.

The session id joins the two halves

It travels in A2A request metadata, never in the prompt text — which is why the agent's tool calls mutate the session the Service will grade, and why prose alone fails a gsm8k task.

6.4 Who Decides What Happens Inside a Task

The sequence of model calls and tool calls is defined nowhere — the model produces it, step by step

component	defines	does NOT define
MCP pod	the task list; each task's initial state and instruction; the TOOL SURFACE — names, schemas, semantics, the observations tools return; the EVALUATOR (evaluate_session → verdict)	any ordering. It answers calls; it never asks for one
agent runtime (in the agent image)	the LOOP — model → tool → observation → model — and when to stop	which tool, with which arguments
the Service	one prompt per task (runner/prompt.py: the task text plus optional context, no tool instructions), the session lifecycle, the per-task timeout, the telemetry	anything about the trajectory
the model	EVERY step	—

Selection is deterministic. Execution is not.

Same task id, same model, same platform, two different legs: tau2 task 2 took 13 chat / 13 tool in one and 9 / 9 in the other; appworld 3d9a636_3 took 20 / 10 and 26 / 13. Of tasks appearing in more than one leg, 8 of 10 tau2 and 3 of 3 appworld differ — the 10 that agree are all gsm8k, where 1 chat / 1 tool leaves nothing to vary.

The stop signal comes from the MCP pod, not from us

No terminal tool name exists anywhere in the Service. The agent reads the tool declarations at connect_mcp, and gsm8k's whole first model call is 320 input tokens — system prompt, task text and every schema. It lands: submit 173/173 on gsm8k, finish 34/35 on appworld (the miss timed out), while tau2 has NO terminal tool — 60/60 end on message.

Why it matters downstream: grading reads the session's FINAL STATE, never a reference trajectory — so a task can pass by two routes at different token cost. Per-task cost and latency are distributions, which is why the reports carry CV columns; and an appworld gap between platforms is not automatically a platform difference.

7.1 The Three Benchmarks — a Difficulty Ladder

Not interchangeable suites: each costs ~an order of magnitude more than the last

	gsm8k	tau2	appworld
MODEL (the default)	Azure/gpt-5-mini	aws/claude-sonnet-5	gemini-2.5-pro
its rate, in / out \$ per 1M	0.25 / 2.00	1.52 / 7.60	1.25 / 10.00
What it tests	multi-step arithmetic	multi-turn dialogue + tools	long-horizon app automation
Task rows measured	172 of 172	60 of 60	35 of 50 (15 timed out)
Rows with lost telemetry	0	0	0
Pass rate	0.97	0.83	0.00
Input tokens / task	341	90,902	269,953
Output tokens / task	180	2,061	25,140
Cost / task, at those rates	\$0.00055	\$0.154	\$0.589
LLM calls / task	1.1	11.4	29.2
Tool calls / task	1.1	11.4	14.6
Median task latency	4.9 s	84 s	264 s
Slowest task seen	39 s	136 s	592 s
Task pool	1,319 (main/test split)	114 (retail domain)	168 (test_normal split)

Read the MODEL row before any dollar figure: the three rungs do not run the same model, and each default is dearer than the one below it — so no cost here transfers to another model. The scale gap is the headline: a tau2 task costs ~267× the input tokens of a gsm8k task, an appworld task ~792× — and in MONEY 279× and 1,069×, because that model change compounds with the token growth (7.6, 7.8). A 50-task gsm8k run is a minute and 2 cents; a 20-task appworld run is 30-45 minutes, millions of tokens and dollars. Budget by benchmark, not by task count.

7.2 What Each Benchmark Stresses

Why all three are in the matrix, and what a result from each does and does not tell you

gsm8k — the canary

- One prompt in, one answer out — no dialogue partner.
- ~1 real LLM call + 1 tool call: the simplest agentic loop.
- Stresses almost nothing about the platform — which is the point.
- A failure here means INFRASTRUCTURE: deploy, auth, LLM reach, telemetry.
- Saturates near 1.0, so it cannot discriminate models. Never read it as one.
- Deterministic enough that task 0 costs 320 input tokens on every cluster — we use that to prove two environments are comparable.
- Cheap enough to run often: a 50-task leg is 25 K tokens — 0.4% of the whole 12-run matrix's bill, and less than a SINGLE appworld task (see 7.5).

tau2 — the discriminator

- Multi-turn: a server-side USER SIMULATOR LLM plays the customer.
- Two models talk to each other, plus ~11 tool calls per task.
- Domain is retail (114 tasks) — the library default, never recorded in artifacts.
- The only leg where model choice dominates: 0.1 with gpt-5-mini vs 0.9 with claude-sonnet-5 on the SAME 10 tasks.
- Episodes are nondeterministic: the same 10 tasks scored 0.90 on one cluster and 1.00 on the other, in the same matrix.
- At n=10 one task moves pass_rate by 0.10 — it cannot resolve less than that.
- 27% of a task's wall time sits INSIDE tool calls — that is the simulator generating, server-side, and its tokens appear in no report of ours.

appworld — the stress test

- Realistic chores across simulated apps: discover APIs, chain many calls.
- ~29 LLM calls, ~15 tool calls, ~270k input, ~4.5 min per task.
- Evaluation is PROGRAMMATIC unit tests over final app state — no LLM judge.
- Pass rate 0.0 is honest, not broken: runs complete and tokens record — the agent just does not finish the job.
- 34 of 35 measured tasks call finish: it believes it is done and the assertions disagree. No verification pass.
- Its job here is pipeline stress: long contexts, big traces, real timeouts.
- Two model calls per tool call — tool shortlisting, which only bites with an API surface this large. The first carries 68% of the input tokens.

7.3 The Three Benchmarks Compared — Task, Purpose, Worth

They differ in the SHAPE of a task, not just its size — which is what makes the ladder a ladder

	gsm8k	tau2	appworld
A task IS	one grade-school word problem, answered in text	one retail customer-service conversation vs a simulated user	one multi-app scenario automated through an API surface
Turn structure	1 model call, ~1 tool call	~11 calls alternating with a USER SIMULATOR in character	~29 calls, each re-sending the whole conversation
Ends when	the answer is emitted	the dialogue resolves, or policy is violated	the goal state is reached — or the 600 s task timeout kills it
Scored by	exact numeric match	task completion + policy compliance (tau2's own scorer)	appworld's state assertions, all-or-nothing
It exists to test	that the PLUMBING works: deploy, auth, telemetry, S3	that the agent HOLDS STATE across turns and uses tools under a policy	that the agent survives LONG HORIZONS: context growth, timeouts, partial failure
A result is worth	a go/no-go on infrastructure. It saturates near 1.0, so it cannot rank models	a genuine model/config comparison — it discriminates, and 0.83 leaves headroom both ways	a stress signal, not a capability score: at 0.00 it tells you what BREAKS, not who is better
Watch out for	1.0 proves nothing about the agent	its user simulator is billed but NOT in our telemetry	15 of 50 tasks time out, and a killed task leaves NO report row

gsm8k is a SMOKE TEST WITH A SCORE — reading its 0.97 as a model measurement is the most common misreading of these numbers. tau2 is the only rung that discriminates, and it fails informatively: a wrong answer, a policy violation and a dropped thread are different failures. appworld earns its place precisely BECAUSE it fails — it is the only leg that has ever exposed a timeout, a context limit or a cache effect before a user did.

7.4 Reading the Numbers — Five Things That Mislead

Every one of these cost us a wrong conclusion first

1. **pass_rate = evaluated_pass / total**

A task that ERRORS before evaluation counts as not passed, so a low rate can mean “failed the task” or “never got judged”. Check the error column too.

2. **The llm column changed meaning between agent versions — never compare across it**

Agents up to exgentic 0.3.5.dev131 issued a max_tokens=1 capability probe counted as a chat span, so THEIR llm=2 on gsm8k means ONE real call. From dev145 the probe is an unbilled GET /v1/models that emits no span — verified absent across all 1,895 chat spans of these two matrices. Current runs count real calls 1:1, with no offset to subtract.

3. **Implausibly small tokens = lost telemetry, and tokens == 0 will not catch it**

Use the structural test: llm <= 1 with tool >= 2 is impossible, since each tool call needs a preceding model turn. A zero-check misses the cases that matter — while the probe existed it was the span that SURVIVED the loss, carrying 8/1 on claude-sonnet-5 and 1/0 on gemini. The structural pair holds across agent versions; it is what the generators use.

4. **Output varies MORE than input, in most runs**

Measured OUT CV > IN CV in 15 of the 22 legs that ran more than one task. On a one-call model gsm8k's prompt is near-constant while answer length swings (IN 0.06-0.09 vs OUT 0.52-0.86); gpt-4.1 needs tool round-trips and varies on the input side too (IN 0.39-0.44). Only long-horizon appworld is input-led, in all four of its legs. Read the CV, not the mechanism.

5. **Task selection is deterministic**

A run takes the first max_tasks tasks, so the same task_id is the same task across runs and clusters, and a smaller run is a prefix of a larger one. That is what makes cross-platform comparison like-for-like — six legs matched to the byte.

7.5 Picking a Benchmark — and What That Leg Costs

The actionable summary: what each benchmark is FOR, and the bill it hands you — in tokens and in dollars

If you want to ...	use	costs
check a cluster / deploy / auth / telemetry path works	gsm8k, 1–10 tasks	< \$0.01
exercise concurrency and volume cheaply	gsm8k, 50 tasks at p=4	\$0.02
compare models meaningfully	tau2 — it discriminates; gsm8k saturates	\$1.70 / 10
stress long contexts, long tasks, timeouts	appworld	\$1.90–2.80 / 5
get a fast signal that nothing regressed	gsm8k — if it fails, fix infrastructure	< \$0.01

Budget by benchmark, not by task count

The eight gsm8k legs together are 0.2% of the matrix's bill on OpenShift (0.4% on KinD). appworld's two legs are 76% of it (57%); tau2's two are 24% (42%). A 50-task gsm8k leg costs 2 cents — less than a twenty-fifth of a SINGLE appworld task.

leg (v1.28)	tok OCP	\$ OCP	tok KinD	\$ KinD
#1 gsm8k, 1 task	470	\$0.0004	790	\$0.0010
#2 gsm8k, 10 tasks	5.1 K	\$0.005	5.1 K	\$0.005
#3 gsm8k, 50 tasks p=4	25 K	\$0.023	24 K	\$0.021
#9 tau2, 10 tasks	1.01 M	\$1.66	1.04 M	\$1.73
#10 tau2, 20 tasks p=4	1.74 M	\$2.90	1.78 M	\$2.94
#11 appworld, 5 tasks	1.49 M	\$2.79	0.93 M	\$1.90
#12 appworld, 20 tasks p=4	5.71 M	\$11.52	2.20 M	\$4.40
all 12 legs	10.0 M	\$18.91	6.0 M	\$11.02

Two legs of the same size are not the same bill

#12 cost 2.6× more on OpenShift than on KinD for the same 20 requested tasks: appworld turn counts are nondeterministic, and the slower cluster's tasks ran longer before the 600 s timeout. So the two \$18.91 / \$11.02 totals are NOT a platform comparison — OpenShift completed 18 appworld tasks against KinD's 10. Size appworld on YOUR cluster.

And the totals UNDERSTATE it three ways: a task killed by the per-task timeout burns tokens but leaves no report row, so appworld's 15 timed-out tasks are missing from the numbers above. tau2's user simulator runs in the MCP pod, which is not instrumented — its inference is billed by the gateway and counted nowhere here. And a leg that replays the gateway's completion cache re-reports stored usage for calls that were never made upstream. 7.7 puts a dollar figure on the fourth: the IBAC judge, which the agent's spans never see.

7.6 The Cost Model — Tokens, Models, Money

What a task costs in tokens AND in dollars, and why the two rankings disagree

per task, pooled	gsm8k	tau2	appworld
MODEL (the default)	gpt-5-mini	claude-sonnet-5	gemini-2.5-pro
its rate, in / out \$ per 1M	0.25 / 2.00	1.52 / 7.60	1.25 / 10.00
LLM calls	1.10	11.38	29.20
input tokens	341	90,902	269,953
output tokens	180	2,061	25,140
total tokens	520	92,963	295,093
× a gsm8k task, in TOKENS	1×	178×	564×
cost at our gateway's rates	\$0.00055	\$0.154	\$0.589
× a gsm8k task, in DOLLARS	1×	279×	1,069×
cost of 100 tasks	\$0.06	\$15.38	\$58.88
input share of tokens	66%	98%	92%
input share of COST	31%	90%	57%
median task latency	4.9 s	84 s	264 s
... of it inside model calls	90%	58%	96%
pass rate	0.97	0.83	0.00

same 5 gsm8k tasks, p=4	gpt-4.1	gpt-5-mini
pass rate (OCP / KiND)	0.80 / 1.00	1.00 / 1.00
LLM calls per task	2.8 – 3.0	1.0
input tokens per task	775 – 837	313
output tokens per task	57 – 63	125 – 368
total tokens per task	832 – 900	438 – 681
cost per task	\$0.0020 – 0.0022	\$0.00033 – 0.00081

Same five tasks: gpt-4.1 costs 4.1× gpt-5-mini

Mean of its 2 legs against gpt-5-mini's 7; leg-to-leg the ratio spans 2.5×–6.7×, because gpt-5-mini's output length swings with reasoning effort. The reasoning model answers in ONE call; gpt-4.1 needs ~3 tool round-trips. On our card the INPUT side decides it alone — 2.6× the tokens at 8× the price is a 21× input bill, and the output side cannot offset it (gpt-5-mini emits 3.6× more output at a quarter the rate, so the two output bills land within 10% of each other).

cost per task = (input tokens × P_in + output tokens × P_out) / 1 M — which model is cheaper is a property of the PRICE LIST, not of the models: compute it for your own rates. Money AMPLIFIES the difficulty ladder rather than tracking it (tau2 178× a gsm8k task in tokens but 279× in dollars; appworld 564× but 1,069×), because the harder benchmarks also run the dearer models — a budget scaled off the token ratios is short by 1.6–1.9×.

7.7 The Rate Card — and Which Way the Dollars Are Wrong

Read off the gateway's admin UI on 2026-09-17 and kept in reference/model_prices.json — posted rates, not an invoice

model (as report.ndjson records it)	provider	in \$/1M	out \$/1M	out/in	used by
Azure/gpt-5-mini-2025-08-07	azure	0.25	2.00	8.0×	gsm8k default
gemini-2.5-pro	vertex_ai	1.25	10.00	8.0×	appworld
aws/claude-sonnet-5	bedrock	1.52	7.60	5.0×	tau2
Azure/gpt-4.1	azure	2.00	8.00	4.0×	leg #4, AND the IBAC judge

Output is priced 4–8× input at every provider

That one fact explains the counter-intuitive results: a reasoning model's verbosity is charged at the expensive end, so a model can win on token count and lose on the bill. Read the input-share-of-COST row, not the share of tokens, to find the driver.

Too LOW: the IBAC judge is not in our telemetry

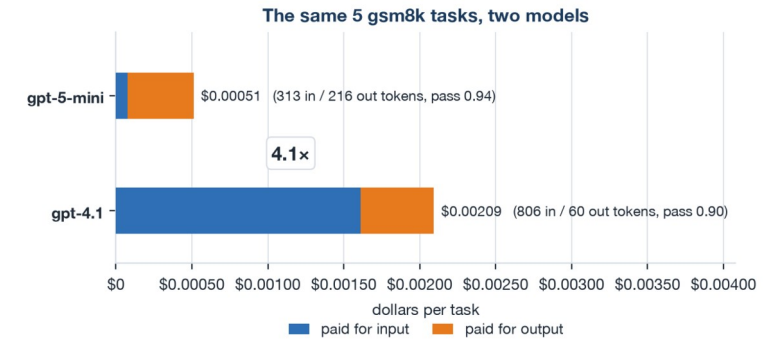
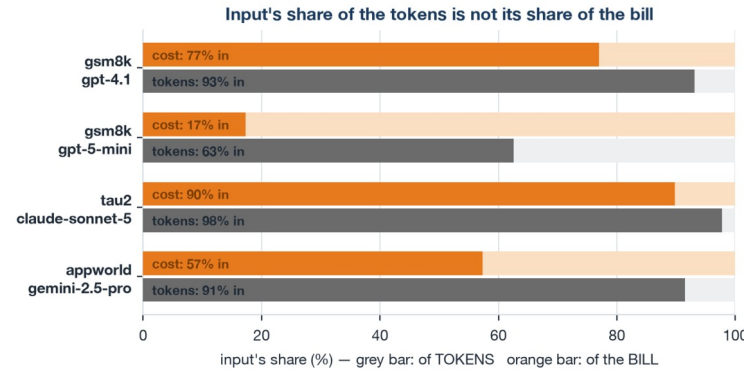
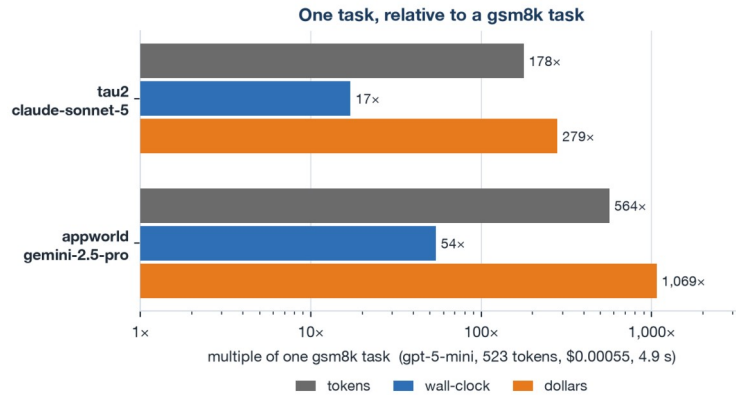
The judge makes ~1 completion per authorized tool call, on Azure/gpt-4.1 — the dearest model on the card — against a FIXED 1,577-char system prompt that does not shrink with the task. That is $\geq \$0.00111$ per call, which is 2.4× the entire gsm8k task it is authorizing (\$0.00045), and 1.4×–4.4× the agent's whole bill across legs #6–#8. On the plugin legs IBAC is not overhead on the bill; it IS the bill. tau2's user simulator is invisible the same way: same model as the agent, but inside the uninstrumented MCP pod.

Too HIGH: an unmodelled cache discount

The gateway REPORTS cached input (usage.prompt_tokens_details.cached_tokens, verified by probe) but publishes no cached-input rate, and report.ndjson stores one undifferentiated llm_input_tokens — so no past run can be re-priced. The exposure is capped by input's share of the bill: if cached input were FREE, tau2 would floor at \$0.0157 a task instead of \$0.154 and appworld at \$0.251 instead of \$0.589, while gsm8k barely moves. That ordering is structural — caching pays off on a long re-sent prefix, which is what makes input dominate.

7.8 Token- and Cost-Efficiency — Where the Money Actually Goes

Measured on the v1.28 pair (267 task rows) at the rate card in 7.7 — one take-away per figure



Money climbs the ladder faster than tokens

Money climbs the difficulty ladder faster than tokens do: tau2 is 178x a gsm8k task in tokens but 279x in dollars, appworld is 564x a gsm8k task in tokens but 1,069x in dollars — so a budget scaled off the token ratios is short by 1.6–1.9x.

Read the share of the BILL, not the share of tokens

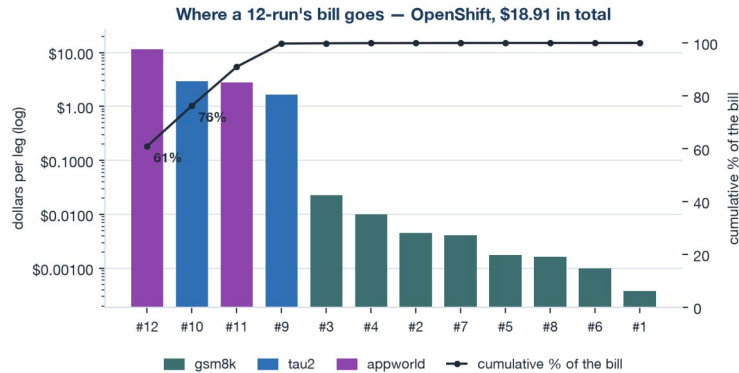
Output is priced 4–8x input on every model we use, so the token split misstates the bill every time: gsm8k on gpt-5-mini is 63% input by tokens and only 17% by cost.

Model choice is settled on the input side

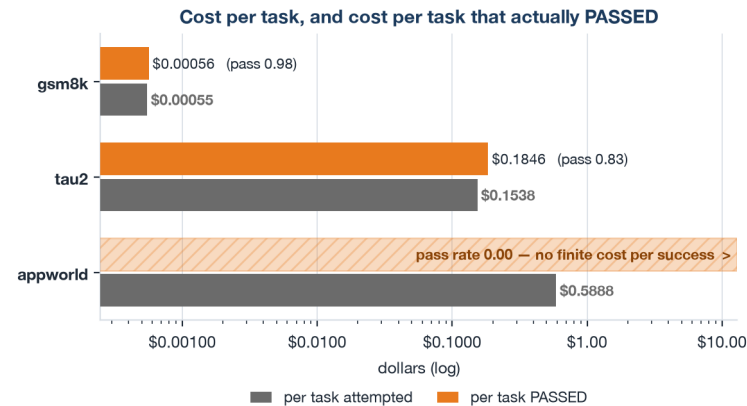
gpt-4.1 costs 4.1x gpt-5-mini on the identical 5 gsm8k tasks, and the input side decides it alone: 21x the input bill, against an output bill the two models split within 10%.

7.9 Token- and Cost-Efficiency — What To Budget, and What To Divide By

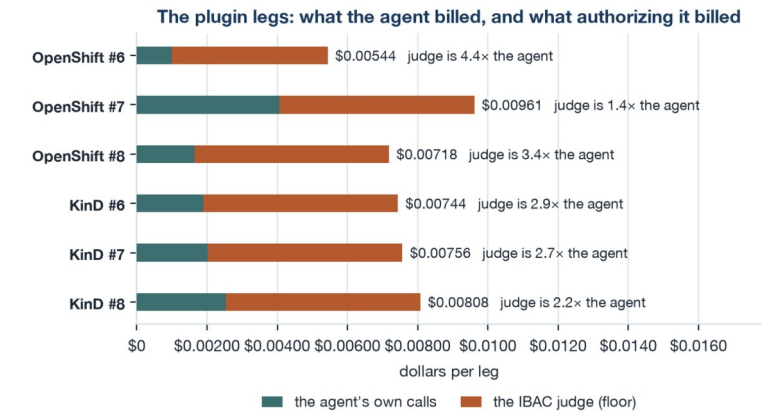
Same three rules every time: budget by benchmark, divide by successes, count what is billed off-telemetry



Two legs of twelve are most of the bill
Two of the twelve legs are 76% of a matrix's bill, while all 8 gsm8k legs together are 0.2% of it — so budget by benchmark, not by task count.



Efficiency only means anything per SUCCESS
Dividing by the pass rate is what turns a token count into an efficiency figure: gsm8k \$0.00055 per task becomes \$0.00056 per PASS; tau2 \$0.1538 per task becomes \$0.1846 per PASS — and appworld has no finite cost per success at all, because it passed nothing.



On the plugin legs, the check outcosts the work
One IBAC judge completion costs at least \$0.00111 — 2.4x the entire gsm8k task on gpt-5-mini it is authorizing — so across legs #6/#7/#8 the judge bills 1.4–4.4x what the agent does.

8.1 The Canonical 12-Run Matrix

One fixed set of 12 request bodies — the same on every platform, every version

#	benchmark	tasks	parallel	what it varies
1	gsm8k	1	1	baseline — the smoke test
2	gsm8k	10	1	volume, still serial
3	gsm8k	50	4	volume + concurrency
4	gsm8k	5	4	model swap → Azure/gpt-4.1
5	gsm8k	5	4	AuthBridge preset: auth-only
6	gsm8k	5	4	AuthBridge preset: ibac-only
7	gsm8k	5	4	AuthBridge preset: full (enforce)
8	gsm8k	5	4	full + per-plugin override ibac:observe
9	tau2	10	1	multi-turn + user simulator
10	tau2	20	4	multi-turn under concurrency
11	appworld	5	1	long-horizon, gemini-2.5-pro
12	appworld	20	4	long-horizon under concurrency

Why a FIXED matrix: task selection is deterministic, so the same leg run anywhere executes the same tasks in the same order. That is what makes a difference attributable to the platform rather than to the workload. Every leg deploys fresh — originally because a warm agent lost telemetry (fixed in agent dev145), now because only a NEWLY CREATED pod re-pulls :latest. Driven by one command (reference/run-12.py): ~1h35m of leg wall time per platform, plus the deploys and ~33 min of gateway-cache gaps. The generated documents in docs/results/ derive every number from the mirrored artifacts.

8.2 The 12 Runs Compared — What Each Band Established

Each leg changes exactly ONE thing against the leg before it, so a difference has one candidate explanation

legs	what they parameterize	what having run them established
#1-#3	gsm8k 1 → 10 → 50 tasks, p=1 → 4 (volume, then concurrency)	the pipeline is stable and DETERMINISTIC: 6 of 12 legs have byte-identical input-token totals across two unlike clusters — the strongest like-for-like check available
#4	gsm8k on Azure/gpt-4.1 (model swap, identical tasks)	the only clean model comparison in the matrix: gpt-4.1 costs 4.1× gpt-5-mini for no pass-rate gain at this difficulty, and the INPUT side decides it alone (7.7)
#5-#8	AuthBridge auth-only / ibac-only / full / full + per-plugin override, same 5 tasks	plugin cost is PLATFORM-SPECIFIC: OpenShift pays in the sidecar (+13.68 s/task), KinD in the judge (+1.54 s) — never quote a per-task overhead without naming the cluster
#9-#10	tau2 10 → 20 tasks, p=1 → 4 (multi-turn, then under load)	multi-turn works end to end, user simulator included — and tau2 is where pass rates carry information (0.75–1.00 across sides)
#11-#12	appworld 5 → 20 tasks, p=1 → 4 (long horizon, then under load)	the limits are real and they are UPSTREAM: 15 timeouts, a 0.00 pass rate, and 76% of the whole matrix's bill in two legs

What the matrix as a whole is worth

Not the pass rates — that the SAME 12 request bodies produce comparable measurements on two unlike clusters. On the v1.28 pair: token capture complete (0 of 267 rows lost their usage span), every task reached the model (0 health-probe losses), 7 of 12 pass rates identical. The 5 that differ are mostly arithmetic on small runs — one task moves a 5-task leg by 0.20.

So read the per-CAUSE table, then the rate

A task lost to a socket and a task lost to a wrong answer land in the same denominator and only one says anything about the agent — which is why every report buckets them (OCP/KinD: transport 1/0, per-task timeout 4/11, upstream agent defect 1/0, wrong answer 2/1). Two further limits: absolute LATENCY does not travel (4–93× apart between our clusters), and the DEPLOY, not the task, is the unit of replication — adding tasks tightens the wrong interval.

8.3 What the 12 Runs Measured

v1.28 on OpenShift (Service on ykt3, workloads on ykt2) — 141 tasks, all 12 legs succeeded

#	bench	pass	err	wall	input tokens
1	gsm8k	1.00	0/1	7 s	320
2	gsm8k	1.00	0/10	41 s	3,166
3	gsm8k	1.00	0/50	46 s	15,684
4	gsm8k	0.80	0/5	16 s	3,874
5	gsm8k	1.00	0/5	25 s	1,564
6	gsm8k	0.60	2/5	39 s	1,258
7	gsm8k	0.80	1/5	30 s	1,564
8	gsm8k	1.00	0/5	23 s	1,564
9	tau2	0.90	0/10	803 s	988,110
10	tau2	0.75	0/20	436 s	1,703,460
11	appworld	0.00	2/5	2433 s	1,387,383
12	appworld	0.00	3/20	1818 s	5,211,584

gsm8k does not quite saturate here: 5 of 8 legs at 1.00

#4's gpt-4.1 swap and #7 lose one task of five, #6 two — five of 86. Read the CAUSE, not the rate: one of #6's was a dropped socket, not the model. The runner retries those now.

tau2 is the leg that moves

0.90 and 0.75 here, 1.00 and 0.80 on KinD. Episodes are nondeterministic: at n=10 one task is worth 0.10, so it cannot resolve less.

appworld 0.00 is the honest result

Runs complete, tokens record, evaluation says the goal was not met — 34 of 35 tasks self-declare finish. The per-task TIMEOUT dominates: 4 of 25 tasks left no report row, so err reads run.json.

0 rows lost attribution, 0 tasks lost to the health probe

The dev145 agent's per-task GET /v1/models probe (10 s cap, no retry) killed 12 of 141 KinD tasks; dev146 fixed it. An earlier matrix's 15 damaged rows passed a tokens == 0 check as clean — hence the structural detector.

9.3M input / 683k output tokens

over 5,719 s of leg wall time — appworld alone is 71% of the input, on 21 of 137 rows.

9.1 OpenShift vs KinD — Like-for-Like

Same 12 request bodies, same Service version, verified-identical instance config

#	bench	pass OCP	pass KinD	in OCP	in KinD	input
1	gsm8k	1.00	1.00	320	320	identical
2	gsm8k	1.00	1.00	3,166	3,166	identical
3	gsm8k	1.00	1.00	15,684	15,684	identical
4	gsm8k	0.80	1.00	3,874	4,183	
5	gsm8k	1.00	1.00	1,564	1,564	identical
6	gsm8k	0.60	0.80	1,258	1,564	
7	gsm8k	0.80	1.00	1,564	1,564	identical
8	gsm8k	1.00	1.00	1,564	1,564	identical
9	tau2	0.90	1.00	988,110	1,019,232	
10	tau2	0.75	0.80	1,703,460	1,743,312	
11	appworld	0.00	0.00	1,387,383	840,030	
12	appworld	0.00	0.00	5,211,584	2,009,348	

7 of 12 pass rates identical, and all 5 deltas favour KinD

So read the per-CAUSE table instead of the rate: wrong answers split 2 (OCP) to 1 (KinD), per-task timeouts 4 to 11 — all appworld — and one dropped socket, OCP #6.

6 legs byte-identical on input tokens

320 / 3,166 / 15,684 / 1,564 x3. Deterministic task selection plus the same model means identical work — the strongest available proof this is a like-for-like comparison, not merely a similar one.

#7 is in that set at 0.80 vs 1.00

Identical input tokens with different pass rates is not a contradiction: same prompts, different answers. Tokens prove the WORK matched; they say nothing about whether it was right.

Wall time: 5,719 s OCP vs 5,975 s KinD

Within 5% overall, but individual legs differ by up to 2.3× in either direction, and appworld — which dominates the total — inverts: #11 is faster on KinD, #12 is slower.

0 lost-attribution rows and 0 probe failures on both sides

137 OCP rows against 130 KinD. The 7-row gap is appworld tasks that timed out before the Service wrote a row (4 OCP, 11 KinD) — not lost telemetry.

9.2 What AuthBridge Costs — and Why It Has No Single Number

The designed 13-leg experiment (n=50, crossover replicates, gateway cache spaced), v1.28

non-LLM s / task (steady)	OpenShift	KinD	OCP step	KinD step
baseline (AuthBridge off)	0.423	0.099	—	—
auth-only	14.101	0.152	+13.68	+0.05
ibac-only	14.148	1.688	+0.05	+1.54
full (auth + ibac)	14.446	1.824	+0.30	+0.14
full + ibac:observe	14.598	1.826	+0.15	+0.00

does the finding travel?	OpenShift	KinD	travels?
Which layer costs anything	the sidecar	the judge	NO
Serial tool calls judged (p=1)	10 / 10	10 / 10	YES
Judged / tool-call ratio, n=50	0.90–0.98	0.90–0.98	YES
tau2 measured / projected	0.31×	2.49×	NO
Between-deploy noise floor	0.22 s	0.155 s	YES
Sidecar CPU / memory	no data	no data	—

The two clusters disagree about WHICH layer costs anything

On OpenShift the whole expense is the sidecar's mere presence and the judge is noise; on KinD the sidecar is nearly free and the judge is all of it. Both are right about their own cluster, and the same condition on the same image measures 4–93× apart. Never quote an absolute per-task plugin figure without naming the cluster.

Not a timeout — we checked

A timeout piles values on a round number. OpenShift's auth-only runs 8.05–17.93 s, SD 1.92, broad and unimodal: contention and queueing. On the quiet cluster the tail moves to full (median 1.82 s, max 14.00 s) because the judge is itself an LLM call — so the sidecar costs reproducibility, not only latency.

One flag decides whether the study measures anything

Every gsm8k leg sends the SAME 50 prompts and the gateway replays completions for ~10 min: unspaced, later legs measure the cache, in run order — the shape of a plugin effect. BM_CACHE_GAP=900 spaces them. The check needs no statistics: the conditions NEST, so no sidecar leg can beat a sidecar-free one. Clear by 41.8× (OCP), 1.4× (KinD).

The deploy is the unit of replication, not the task

A per-task interval measures variance WITHIN one deploy. The honest floor is between two deploys of one condition, and it exceeds every step but the dominant layer — so presets cannot be RANKED at any task count; add deploys, not tasks. Cut the other way: at $\sigma \approx 0.23$ s two deploys already resolve a 1 s effect, so the small steps are genuinely small rather than unresolved.

10. What the Service Can & Cannot Enact

The HTTP-only boundary, made explicit

✓ Enacts over HTTP

- Deploy MCP tool + A2A agent (CPU/mem, image, env)
- Deploy-time model swap (per-experiment agent)
- authbridge_enabled → inject the sidecar (layer-2)
- plugin_preset / plugins / on_error → AuthBridge layer-3
→ pluginPreset/plugins/onError; operator renders ConfigMap
- Run benchmark sessions; collect pass/fail + latency
- Emit + read MLflow traces; export to S3
- Service-owned config: MLflow + S3 via PUT /config
benchmarker only

✗ Out-of-band (reports, doesn't do)

- Cluster Secrets (hf-secret, openai-secret)
operator provisions; run precheck returns 424 naming it
- AuthBridge CLUSTER config — ibac judgeEndpoint / judgeModel
in the platform-config ConfigMap, not the agent API
- Any cluster-level API call — Rossoctl does these server-side

Principle:

- Never silently ignore an un-enactable request —
precheck (424) or reject (422), with an actionable reason

Cluster-agnostic by construction: works on kind / vanilla k8s / OpenShift; cross-cluster runs use per-instance route templates + a reachable internal issuer.